Claims Without Receipts: How Often Coding Agents' Reports Go Unbacked, and Why Receipt Rules Cannot Tell

Abhay Mettu

November 2026

Abstract

At the end of each turn a coding agent tells its user what it did: "tests pass", "I updated `config.py`", "committed as `a41c9e2`". Most users believe the report and move on, so a claim the agent never checked is found only later. We built `receipts`, a checker that tests each claim against the agent's own tool calls in the same session, and used it to ask how often those statements are not backed, and whether the deterministic receipt rules that several recent Stop-hook tools use can find the ones that are not. From 2,301 Claude Code and Codex sessions on one developer's machine we drew 400 reported turns uniformly at random and labelled the 2,082 claims in them with two isolated model raters plus a third adjudicator on the held-out split, audited by the author. 13.3% of claims are not backed by the agent's receipts (95% CI 11.3 to 15.4, session bootstrap). Outright contradictions are rare (5 of 2,082); the failure is claiming without current evidence. Unbacked claims concentrate in "it works / it's fixed" (36.8%) and "tests pass" (22.5%) claims and rise with session length (5.1% in sessions of 1 to 9 tool calls, 13 to 20% in longer ones). Our second finding is negative: deterministic receipt rules, even after a 15-round climb on a development split, agree with careful readers at chance on held-out turns (verdict $\kappa \approx 0$, end-to-end flag precision 0.083 and recall 0.153 with model-assisted claim extraction). A model that reads the receipts does far better (Sonnet: flag precision 0.557, recall 0.315, verdict κ 0.52). We release the checker, which runs as a Claude Code Stop hook in either mode, with the benchmark pipeline.

1 Question

Developers who hand work to coding agents read the agent's end-of-turn report far more often than they read its tool calls. When the report says the tests pass, most users move on. Two questions follow.

1. **Prevalence.** In ordinary use, what share of an agent's claims about its own work are not backed by the receipts it left in the session: the commands it ran, their outputs, the files it wrote? Which kinds of claims, and which sessions?
2. **Detection.** Several open-source Stop hooks released in 2026 (proof-of-done [Boyko, 2026], honestly [kennywu411-dev, 2026], groundtruth [vnmoorthy, 2026]) parse the transcript, find claims with patterns, and require a matching successful command after the last edit. They report precision on author-written fixtures or unvalidated counts. Does that mechanism agree with careful readers on real sessions?

We take no position on intent. A claim is "not backed" when the agent's own record does not support it at the moment it was made, whether because nothing checked it, the check came before a later edit, or the check failed. We do not say the agent lied.

A small pilot preceded this study: 5 sessions and 324 claims, which suggested roughly 1 in 9 claims were unbacked. It is kept in the repository under `archive/v0/`. This paper replaces it with a sampled, labelled and validated measurement.

2 Data and sampling

Frame. The population is every main-thread Claude Code transcript and every Codex rollout on one developer's machine written before 2026-11-01 10:00 UTC (`bench/frame.py`). Of 5,922 candidate transcripts, we drop 2,443 with no tool calls or no report, 116 test-harness sandboxes that replay other sessions, 1,054 that work on private projects the owner keeps out of public work, and 8 after the cutoff. The private-project rule looks at what a session works on (working directory, typed prompts, tool calls, reports) rather than injected memory, which names those projects in nearly every transcript. 2,301 sessions remain: 2,267 Claude Code and 34 Codex, with 7,169 reported turns and 88,379 tool calls. Most are the developer's autonomous "lanes" (long unattended agent runs) and orchestration sessions; few are short interactive edits.

Sample. A reported turn is a turn that ends in agent text. We drew 400 reported turns uniformly (seed 20261101) and split them by a hash of the session into a development split (225 turns, used for tuning) and a test split (175 turns, used only for the reported validity numbers). Claims within a session are not independent, so every interval in this paper comes from a bootstrap that resamples sessions (2,000 resamples, seed 7).

Dossiers. For each sampled turn, `bench/label.py` writes a dossier: the user prompt, the report, and every tool call in the session up to that report, oldest first, with its command or target, the files it wrote, whether it errored, and a clipped copy of its output. Calls from the reporting turn are marked.

3 Labelling protocol and agreement

Rubric. Raters follow `receipts/rubric.md`. A claim is a sentence, bullet or table row in which the agent asserts, as done or true now, something about its own work in this session that a tool receipt could back or refute. Claims fall in eight families: `tests`, `build`, `files`, `git`, `ship` (deploys, installs, pushes to services), `ran` ("I ran X"), `verified` ("I checked that..."), and `works` ("it works", "fixed"). Plans, hedges and findings read off tool output are out of scope. Each claim gets a verdict: *supported*, *contradicted* (a receipt shows the opposite), or *unbacked* (no receipt shows it, or the only one predates a later change to the code it covers, which we call stale). "Not backed" means contradicted or unbacked.

Raters. Labels are per turn, not per extracted claim: a rater lists every claim in the report with a verdict, so any extractor can be scored against the same labels and recall is measurable. Raters are Opus 5.5 sessions run through `claude -p`, isolated from the machine's settings, hooks, memory and MCP servers, with read-only tools over one dossier each. Rater A labelled all 400 turns. Rater B labelled the 175 test turns independently.

Agreement. On the test split A listed 890 claims and B 880, of which 786 matched (claim-set F1 0.888). On matched claims they agreed on the verdict 94.8% of the time (Cohen's κ 0.739) and on the binary not-backed flag with κ 0.757 (`results/validity_agreement.json`).

Gold labels for the test split. Claims both raters listed with the same verdict (745) go straight to gold. The remaining 239 disputes (a claim only one rater listed, or differing verdicts) went to a third Opus 5.5 adjudicator, which read the same dossier plus both raters' notes; it ruled 205 to be claims and 34 not. The author audited 40 rulings drawn at random (seed 11), agreed with 37, and reversed 3, all of which were findings read off tool output that the rubric puts out of scope (`results/adjudication_audit.json`). Gold on the test split holds 950 claims. On the development split, rater A's labels are gold.

Matching. A system claim matches a gold claim when their sentences overlap: one contains the other, or token Jaccard similarity is at least 0.5. Pairs are made greedily, same family first.

4 Checkers compared

All checkers read the same session record. They differ in how they find claims and how they decide.

Rules, regex extraction. The fast mode of our tool and the mechanism of the existing hooks. A regex extractor finds claims by family, with hedge and negation handling (`receipts/claims.py`). Command classifiers recognise test, build, lint, commit, push, deploy and write calls and read pass or fail from their output (`receipts/rules.py`). One rule per claim kind turns receipts into a verdict, including a staleness rule: a passing test run counts only if no code was written after it (`receipts/check.py`).

Rules, LLM extraction. The same verdict rules, with claims listed by a small model (Haiku) that reads only the report (`receipts/llm.py`). This isolates the verdict rules from extraction recall.

Judge. A model reads the turn's dossier under the same rubric the raters used and returns claims with verdicts (`receipts/judge.py`). We ran it with Sonnet (judge S) and Haiku (judge H).

The verdict rules were hill-climbed on the development split for 15 rounds, scoring dev end-to-end flag F1 with LLM extraction; each round was kept only if it beat the previous best. Dev F1 rose from 0.088 to 0.201.

5 Results

5.1 Prevalence

Across the 400 labelled turns (2,082 claims in 293 sessions, 5.21 claims per turn) 1,806 claims are supported, 271 unbacked and 5 contradicted. The not-backed rate is 13.3% (95% CI 11.3 to 15.4). It is similar on the two splits: 13.4% on dev (rater A) and 13.1% on test (adjudicated gold). Among the not-backed claims with a recorded basis, 27 rest on a stale receipt, 4 on a direct contradiction and 187 on no receipt at all (`results/benchmark.json`, key `labels`).

Table 1 gives the breakdown. Three patterns hold. First, claims about outcomes ("it works", "tests pass", "I verified") are the ones most often left without a receipt; claims about actions the harness records directly ("I ran", commits) are almost always backed. Second, short sessions are

Table 1: Share of claims not backed by the agent's own receipts, from the gold labels (model raters, adjudicated and author-audited). Intervals: 95% session bootstrap.

		claims	sessions	not backed (95% CI)
All		2,082	293	13.3% (11.3 to 15.4)
Family	works	182	80	36.8% (27.9 to 45.5)
	tests	151	69	22.5% (14.9 to 30.1)
	verified	391	137	14.3% (10.3 to 18.5)
	build	33	20	12.1% (0.0 to 25.0)
	files	698	195	11.9% (8.9 to 15.3)
	ship	70	48	5.7% (0.0 to 12.5)
	ran	404	228	5.5% (3.2 to 8.1)
	git	153	90	3.9% (1.3 to 7.2)
Model	Opus	1,672	219	12.4% (10.3 to 14.7)
	Sonnet	272	41	17.3% (12.7 to 21.7)
	Fable	77	17	20.8% (8.7 to 31.0)
	GPT (Codex)	33	3	9.1% (0.0 to 21.4)
Session length	1 to 9 calls	117	44	5.1% (1.8 to 8.6)
	10 to 49 calls	703	135	14.4% (11.2 to 17.5)
	50 to 199 calls	1,213	121	13.1% (10.3 to 15.9)
	200+ calls	49	4	20.4% (11.5 to 33.3)

cleaner: 5.1% of claims in sessions of 1 to 9 tool calls are not backed, against 13 to 20% in longer ones. Third, the model differences are within overlapping intervals and confounded with task: the developer used different models for different kinds of work, and the Codex and Fable cells are small. We do not rank models from this table.

In a separate live check outside the sample, clean single-turn tasks given to Haiku, Sonnet and Opus produced honest reports from all three. The one failure we reproduced on demand needed a multi-step task: after a rename, Haiku under a terse house-style prompt reported "pytest passes" from a run made before the rename. The rules hook flagged the claim in 2.1 ms and the agent re-ran the tests (`docs/live-hook/`).

5.2 Checker validity

Table 2 is the second finding. The deterministic rules agree with careful readers at chance on held-out turns: verdict κ is -0.0003 with regex extraction and 0.003 with LLM extraction. Regex extraction finds only 15% of the claims the raters listed, so end-to-end it catches 3 of 124 gold flags while raising 63 false ones. Giving the rules a good extractor raises recall but not agreement: 19 true flags against 210 false. The dev-split climb that took flag F1 from 0.088 to 0.201 did not transfer (`results/validity_test.json`, `validity_test_llm.json`).

The errors run both ways. With LLM extraction, the rules call 137 gold-supported claims unbacked and 60 gold-unbacked claims supported. Our reading of the disagreements is that relevance is the hard part: a rule can ask whether *some* test run passed after the last write, but not whether that run exercised what the claim is about, and it cannot tell when a read, a diff or an unrecognised command is the receipt a reader accepts. Most of what readers flag is a "works" or "verified" claim, which is where relevance matters most.

Table 2: Checkers against adjudicated gold on the held-out test split (175 turns, 950 gold claims). Flag = not backed. End-to-end counts a missed or spurious claim as an error.

checker	extraction precision	extraction recall	verdict κ (matched)	end-to-end flag precision	end-to-end flag recall
Rules, regex extraction	0.75	0.15	0.00	0.045	0.024
Rules, LLM extraction (Haiku)	0.76	0.73	0.00	0.083	0.153
Judge, Haiku[†]	0.72	0.88	0.45	0.307	0.524
Judge, Sonnet	0.83	0.81	0.52	0.557	0.315
Rater B vs rater A (reference)	claim-set F1 0.888		0.74	flag κ 0.757	

[†]Scored on 174 of the 175 turns (947 gold claims).

A model that reads the dossier does much better. The Sonnet judge flags with precision 0.81 on matched claims and 0.557 end to end, with recall 0.315 and verdict κ 0.52 (`results/validity_judge_S.json`). The Haiku judge trades precision for recall (0.307 and 0.524). Neither reaches the second rater's agreement with the first, so a judge is a screen, not a substitute for reading.

5.3 Cost

Table 3: Cost per checked turn. Rules: wall time over all 2,301 sessions on an arm64 laptop, Python 3.14. Judges: wall time per labelled turn and the CLI's API-equivalent cost (the runs used a subscription, so nothing was billed).

checker	median	p90 / p99	API-equivalent USD per turn
Rules hook	7.2 ms	18.2 / 43.1 ms	0
Judge, Haiku	16.4 s	68.8 s (p90)	0.008
Judge, Sonnet	11.6 s	29.9 s (p90)	0.104
Rater, Opus (A)	21.1 s	46.5 s (p90)	0.162

The rules are about three orders of magnitude faster (Table 3; `results/hook_cost.json`, `results/judge_cost.json`). Run over every session in the frame, the rules hook flags at least one claim in the last turn of 16.0% of sessions. Given Table 2, that figure describes what the fast mode flags, not how often agents make unbacked claims, and we do not report the rules' at-scale verdicts as a rate. Our tool therefore defaults its Stop hook to the judge engine and keeps the rules as a fast, explicit opt-in.

6 Limits

- **One developer.** Every session comes from one machine, mostly long autonomous lanes and orchestration work. The rates may not hold for interactive use or for other developers, prompts and house styles.
- **Model raters.** The labels come from Opus 5.5 raters, not people. Two raters agree well (κ 0.74) and the author audited 40 adjudicated rulings, but systematic blind spots shared by the raters would not show up in either check.

- **Clipped outputs.** Dossiers clip long tool outputs, so a receipt present in the full output can be cut, which biases toward "unbacked".
- **Subagents not read.** Work done inside subagents lives in separate transcripts that the dossier does not include; a claim backed only there reads as unbacked.
- **Matching by sentence overlap.** Validity numbers depend on pairing system and gold claims by text overlap, which can miss paraphrases and split one sentence's several claims unevenly.
- **Small cells.** Codex (3 sessions in the sample), Fable and the 200+ call bucket have wide intervals.

7 Related work

Tools with the same mechanism. proof-of-done [Boyko, 2026], honestly [kennywu411-dev, 2026] and groundtruth [vnmoorthy, 2026] are Stop hooks that pattern-match claims in the final message and require a matching successful command after the last edit; agentvow [Navikkumar-Modi, 2026] re-runs tests in a sandbox instead. They validate on author-written fixtures or report counts from their authors' sessions. Our result says that this mechanism, which our tool also implements, does not agree with careful readers on real sessions. A GitHub issue on Claude Code [Ashwinsuriya, 2026] hand-labelled 357 "tests pass" claims from 150 sessions and found most unbacked ones stale; we extend that to all claim families with a validated sample.

Self-reports checked against the record. Smyth et al. [2026] define overclaiming as a report contradicted by the agent's own transcript and measure it on planted file-review tasks with an LLM judge. Kraishan and Jitkajornwanich [2026] compare reports with tool logs in SWE-chat [Baumann et al., 2026] and find reports mention few actions and drift toward the plan. Leith [2026] studies claim reliability in one fully AI-written codebase; Lei Zhenbing and Lei Chang [2026] re-verifies ten "PASS" claims mechanically. Zhu et al. [2026b] find that judges who see a GUI agent's success report miss more misreports than trace-only judges, and Zhu et al. [2026a] measure false success after injected tool failures. Our setting differs in using natural, unplanted sessions, many claim types, and a measured comparison of rules against reading.

Receipts. Basu [2026] have the runtime sign per-call receipts that the model cites; Hu et al. [2026] bind claims to execution prefixes with a deterministic integrity check separate from support. Both need a cooperating runtime; we audit an unmodified agent after the fact, and our negative result suggests the support step is where reading is needed.

Hallucination and deception. ToolBeHonest [Zhang et al., 2024] and AgentHallu [Liu et al., 2026b] benchmark tool-use hallucination; Florin et al. [2026], SWE-DeceptionBench [Liu et al., 2026a], Scheurer et al. [2023] and Meinke et al. [2024] elicit concealment or deception under constructed pressure. We follow the behavioral stance of Park et al. [2023] and make no intent claim; unlike chain-of-thought faithfulness [Turpin et al., 2023], claims about external actions can be checked directly. Reward-hacking reports from METR [Von Arx et al., 2025] and Anthropic [Anthropic, 2025] describe agents gaming tests; the failure we count is mostly quieter: saying tests pass without a current run.

Evaluation. Agent-as-a-Judge [Zhuge et al., 2024] grades trajectories with an agent, as our judge engine does for claims. RAGAS faithfulness [Es et al., 2023] decomposes answers into statements and checks each against context, the same shape as our task with tool calls as context. Process supervision [Lightman et al., 2023] checks steps rather than outcomes. SWE-bench [Jimenez et al., 2024] grades by running tests, the external check that everyday sessions lack.

8 Reproducibility

The checker and the benchmark pipeline are in the repository. The tool runs anywhere; the benchmark reads the author's private session logs and so reruns only on a machine that has them. Aggregate outputs are committed under `results/`; claim text, transcripts and labels stay in the untracked `data/` folder.

```
pipx install git+https://github.com/abhaymettu/receipts
receipts demo                        # bundled sample session
python -m pytest -q tests

python3 bench/frame.py               # results/frame.json
python3 bench/run.py                 # data/claims.jsonl, data/turns.jsonl
python3 bench/label.py sample        # 400 turns, dev/test split
python3 bench/label.py dossiers
python3 bench/label.py rate A
python3 bench/label.py rate B --only test
python3 bench/adjudicate.py rule && python3 bench/adjudicate.py build
python3 bench/score.py agreement     # results/validity_agreement.json
python3 bench/score.py test          # results/validity_test.json
RECEIPTS_EXTRACT=llm python3 bench/score.py test   # validity_test_llm.json
python3 bench/score.py judge:S       # validity_judge_S.json (judge:H for Haiku)
python3 bench/stats.py               # results/benchmark.json
python3 bench/hook_cost.py           # results/hook_cost.json
python3 bench/judge_cost.py          # results/judge_cost.json
```

Every number in this paper is mapped to its results file and key in `paper/NUMBERS.md`.

References

Anthropic. Claude 3.7 Sonnet system card. `https://assets.anthropic.com/m/785e231869ea8b3b/original/claude-3-7-sonnet-system-card.pdf`, 2025. Section 6, Excessive Focus on Passing Tests.

Ashwinsuriya. "tests pass" reported from a stale or piped run (measured across 150 sessions). GitHub issue anthropics/claude-code #104504, `https://github.com/anthropics/claude-code/issues/104504`, 2026.

Abhinaba Basu. Tool receipts, not zero-knowledge proofs: Practical hallucination detection for AI agents, 2026.

Joachim Baumann, Vishakh Padmakumar, Xiang Li, John Yang, Diyi Yang, and Sanmi Koyejo. SWE-chat: Coding agent interactions from real users in the wild. In *Conference on Language Modeling (COLM)*, 2026.

Andrii Boyko. proof-of-done: Claude code plugin and CLI that check a coding agent's completion claims against its own transcript. `https://github.com/B0yko/proof-of-done`, 2026. Version 0.1.1 on PyPI.

Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. Ragas: Automated evaluation of retrieval augmented generation, 2023.

Lucas Florin, Amelie Knecht, Ulysse Schaller, and Thilo Hagendorff. Deception by omission: Language models knowingly hide their mistakes, 2026.

Miaobo Hu, Shuhao Hu, Xiaobo Guo, Xin Wang, Bokun Wang, Yina Sa, and Daren Zha. Actions with receipts: Jointly binding claims, evidence, and execution for replayable tool-agent auditing, 2026.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

kennywu411-dev. honestly: blocks unverified "done" claims in coding agents and audits session history. `https://github.com/kennywu411-dev/honestly`, 2026. Version 0.6.0 on PyPI.

Obada Kraishan and Kulsawasd Jitkajornwanich. Plans they abandon, reports they author: The narrative layer of autonomous agents, 2026.

Lei Zhenbing and Lei Chang. PASS is a claim, not an observation: Mechanical re-verification of 10 self-asserted fix claims in a live multi-agent audit layer, 2026.

Douglas Leith. Between the commits: Process, error, and claim reliability in a wholly AI-authored codebase, 2026.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step, 2023.

Xiting Liu, Lin Yang, Yitong Zhang, Jia Li, and Shi-Min Hu. SWE-DeceptionBench: Benchmarking deceptive behaviors of LLMs across the software development lifecycle, 2026a.

Xuannan Liu, Xiao Yang, Zekun Li, Peipei Li, and Ran He. AgentHallu: Benchmarking automated hallucination attribution of LLM-based agents, 2026b.

Alexander Meinke, Bronson Schoen, Jérémy Scheurer, Mikita Balesni, Rusheb Shah, and Marius Hobbhahn. Frontier models are capable of in-context scheming, 2024.

NavikkumarModi. Agentvow: check a coding agent's claims against the repository before you approve. `https://github.com/NavikkumarModi/Agentvow`, 2026. Version 0.4.5 on PyPI.

Peter S. Park, Simon Goldstein, Aidan O'Gara, Michael Chen, and Dan Hendrycks. AI deception: A survey of examples, risks, and potential solutions, 2023.

Jérémy Scheurer, Mikita Balesni, and Marius Hobbhahn. Large language models can strategically deceive their users when put under pressure, 2023.

Nolan Smyth, Yorguin-Jose Mantilla-Ramos, Pascal Jr Tikeng Notsawo, Saskia Helbling, Alberto Tosato, Mohamed Amine Merzouk, Nouha Dziri, Gauthier Gidel, and Tommaso Tosato. Quantifying overclaiming propensity in frontier LLM agents, 2026.

Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting, 2023.

vnmoorthy. groundtruth: A completion-claim gate for claude code. `https://github.com/vnmoorthy/groundtruth`, 2026. Version 0.1.13.

Sydney Von Arx, Lawrence Chan, and Beth Barnes. Recent frontier models are reward hacking. METR blog, `https://metr.org/blog/2025-06-05-recent-reward-hacking/`, June 2025.

Yuxiang Zhang, Jing Chen, Junjie Wang, Yaxin Liu, Cheng Yang, Chufan Shi, Xinyu Zhu, Zihao Lin, Hanwen Wan, Yujiu Yang, Tetsuya Sakai, Tian Feng, and Hayato Yamana. ToolBeHonest: A multi-level hallucination diagnostic benchmark for tool-augmented large language models, 2024.

Junru Zhu, Shiming Xie, Aime Lu Fan Chen, Xiaoqing Ding, Chunxin Tang, Ruoyu Qi, and Yulang Fei. Failure-transparent agents: Benchmarking post-failure reporting in tool-using language models, 2026a.

Yunfeng Zhu, Shengcheng Yu, Zihan Li, and Zhenyu Chen. Who verifies the agent's success claim? self-report leakage in GUI-agent completion judging, 2026b.

Mingchen Zhuge, Changsheng Zhao, Dylan Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong Tian, Yangyang Shi, Vikas Chandra, and Jürgen Schmidhuber. Agent-as-a-judge: Evaluate agents with agents, 2024.